

## SKILL WEEK-10:

TASK-1: To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test 4 Signal Events

Ans:

```
GNU nano 8.7.1 task1_signal_handlers.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>

volatile sig_atomic_t signal_count = 0;

void handle_signal(int sig)
{
    signal_count++;

    if (sig == SIGINT)
        write(STDOUT_FILENO, "\nReceived SIGINT (Ctrl+C)\n", 26);

    else if (sig == SIGTERM)
        write(STDOUT_FILENO, "\nReceived SIGTERM\n", 18);

    else if (sig == SIGUSR1)
        write(STDOUT_FILENO, "\nReceived SIGUSR1\n", 18);

    else if (sig == SIGALRM)
        write(STDOUT_FILENO, "\nReceived SIGALRM\n", 18);
}

int main()
{
    printf("TASK-1: Signal Handler and Signal Action Test\n");
    printf("PID: %d\n", getpid());

    struct sigaction sa;

    memset(&sa, 0, sizeof(sa));
```

```

struct sigaction sa;

memset(&sa, 0, sizeof(sa));
sa.sa_handler = handle_signal;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;

/*
 * Register signal handlers.
 */
sigaction(SIGINT, &sa, NULL);
sigaction(SIGTERM, &sa, NULL);
sigaction(SIGUSR1, &sa, NULL);
sigaction(SIGALRM, &sa, NULL);

printf("\nFour signal handlers registered:\n");
printf("1. SIGINT\n");
printf("2. SIGTERM\n");
printf("3. SIGUSR1\n");
printf("4. SIGALRM\n");

/*
 * Test SIGALRM automatically.
 */
alarm(3);

printf("\nWaiting for signals...\n");
printf("Press Ctrl+C to test SIGINT.\n");

for (int i = 1; i <= 10; i++) {
    printf("Program running... %d/10\n", i);
    sleep(1);
}

```

```

for (int i = 1; i <= 10; i++) {
    printf("Program running... %d/10\n", i);
    sleep(1);
}

printf("\nTotal signals handled: %d\n", signal_count);
printf("Signal handling test completed.\n");

return 0;
}

```

```

arikel@DESKTOP-AMUSJME:~/week5prac$ nano task1_signal_handlers.c
arikel@DESKTOP-AMUSJME:~/week5prac$ gcc task1_signal_handlers.c
-o task1_signal_handlers
arikel@DESKTOP-AMUSJME:~/week5prac$ ./task1_signal_handlers
TASK-1: Signal Handler and Signal Action Test
PID: 1119

Four signal handlers registered:
1. SIGINT
2. SIGTERM
3. SIGUSR1
4. SIGALRM

Waiting for signals...
Press Ctrl+C to test SIGINT.
Program running... 1/10
Program running... 2/10
Program running... 3/10

Received SIGALRM
Program running... 4/10
Program running... 5/10
Program running... 6/10
Program running... 7/10
Program running... 8/10
Program running... 9/10
Program running... 10/10

Total signals handled: 1
Signal handling test completed.

```

Conclusion: Implemented signal handlers using `sigaction()` and configured signal actions for SIGINT, SIGTERM, SIGUSR1 and SIGALRM. The program manages signal context using a signal mask and safely records received signals using `sig_atomic_t`. The four signal events were tested to verify stable signal handling without unexpected program termination.

TASK-2: To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt 4 Handling

Ans:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t foreground_pid = -1;

void handle_sigint(int sig)
{
    printf("\nShell received SIGINT.\n");

    if (foreground_pid > 0) {
        printf("Forwarding SIGINT to foreground job PID %d\n",
            foreground_pid);

        kill(foreground_pid, SIGINT);
    } else {
        printf("No foreground job is running.\n");
    }
}

int main()
{
    printf("TASK-2: SIGINT Handling and Foreground Job Control\n");
    printf("Shell PID: %d\n", getpid());

    /*
     * Protect the shell from default SIGINT termination.
     */
    struct sigaction sa;

    sa.sa_handler = handle_sigint;
```

```

    */
    struct sigaction sa;

    sa.sa_handler = handle_sigint;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    sigaction(SIGINT, &sa, NULL);

    printf("\nStarting foreground job...\n");

    foreground_pid = fork();

    if (foreground_pid < 0) {
        perror("fork");
        return 1;
    }

    if (foreground_pid == 0) {

        printf("Foreground job started.\n");
        printf("Job PID: %d\n", getpid());

        for (int i = 1; i <= 10; i++) {
            printf("Foreground job running... %d/10\n", i);
            sleep(1);
        }

        printf("Foreground job completed normally.\n");
        exit(0);
    }

    printf("Foreground job PID = %d\n", foreground_pid);

```

```

    for (int i = 1; i <= 10; i++) {
        printf("Foreground job running... %d/10\n", i);
        sleep(1);
    }

    printf("Foreground job completed normally.\n");
    exit(0);
}

printf("Foreground job PID = %d\n", foreground_pid);
printf("Press Ctrl+C to interrupt the foreground job.\n");

int status;

waitpid(foreground_pid, &status, 0);

if (WIFEXITED(status)) {
    printf("\nJob state: Completed\n");
}
else if (WIFSIGNALED(status)) {
    printf("\nJob state: Terminated by signal %d\n",
           WTERMSIG(status));
}

foreground_pid = -1;

printf("Shell remains active.\n");
printf("SIGINT handling test completed.\n");

return 0;
}
|

```

Compile and run:

```
arikela@DESKTOP-AMUSJME: x + v - □ ×
arikela@DESKTOP-AMUSJME:~/week5prac$ ./task2_sigint_shell
TASK-2: SIGINT Handling and Foreground Job Control
Shell PID: 1228

Starting foreground job...
Foreground job PID = 1229
Press Ctrl+C to interrupt the foreground job.
Foreground job started.
Job PID: 1229
Foreground job running... 1/10
Foreground job running... 2/10
^C
Shell received SIGINT.
No foreground job is running.
Foreground job running... 3/10

Shell received SIGINT.
Forwarding SIGINT to foreground job PID 1229

Job state: Completed
Shell remains active.
SIGINT handling test completed.

Shell received SIGINT.
No foreground job is running.
Foreground job running... 4/10
arikela@DESKTOP-AMUSJME:~/week5prac$ Foreground job running... 5
/10
Foreground job running... 6/10
Foreground job running... 7/10
Foreground job running... 8/10
Foreground job running... 9/10
^C
arikela@DESKTOP-AMUSJME:~/week5prac$ Foreground job running... 1
0/10
Foreground job completed normally.
```

Conclusion: Implemented SIGINT handling for a shell-like parent process. The shell registers a SIGINT handler and maintains the foreground process PID. When Ctrl+C generates SIGINT, the shell captures it and forwards the signal to the foreground process using kill(). The shell remains active while the foreground process is terminated, and waitpid() updates the job state

TASK-3: To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow

Ans:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t foreground_pid = -1;

void handle_sigtstp(int sig)
{
    printf("\nShell received SIGTSTP.\n");

    if (foreground_pid > 0) {
        printf("Suspending foreground job PID %d\n",
            foreground_pid);

        kill(foreground_pid, SIGSTOP);
    }
}

int main()
{
    printf("TASK-3: SIGTSTP Job Suspension and Recovery\n");
    printf("Shell PID: %d\n", getpid());

    struct sigaction sa;

    sa.sa_handler = handle_sigtstp;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    sigaction(SIGTSTP, &sa, NULL);
```

```

sigaction(SIGTSTP, &sa, NULL);

foreground_pid = fork();

if (foreground_pid < 0) {
    perror("fork");
    return 1;
}

if (foreground_pid == 0) {

    printf("Foreground job started.\n");
    printf("Job PID: %d\n", getpid());

    for (int i = 1; i <= 20; i++) {
        printf("Job running... %d/20\n", i);
        sleep(1);
    }

    printf("Job completed.\n");
    exit(0);
}

printf("Foreground job PID = %d\n", foreground_pid);
printf("Press Ctrl+Z to suspend the job.\n");

int status;

/*
 * Wait until job stops or completes.
 */
waitpid(foreground_pid, &status, WUNTRACED);

```

```

waitpid(foreground_pid, &status, WUNTRACED);

if (WIFSTOPPED(status)) {

    printf("\nJob state updated: STOPPED\n");

    printf("Job is suspended.\n");
    printf("Press ENTER to resume it.\n");

    getchar();

    printf("Sending SIGCONT...\n");

    kill(foreground_pid, SIGCONT);

    printf("Job state updated: RUNNING\n");

    /*
     * Continue waiting for completion.
     */
    waitpid(foreground_pid, &status, 0);
}

if (WIFEXITED(status)) {
    printf("\nJob state updated: COMPLETED\n");
}
else if (WIFSIGNALED(status)) {
    printf("\nJob terminated by signal %d\n",
           WTERMSIG(status));
}

```

```

else if (WIFSIGNALED(status)) {
    printf("\nJob terminated by signal %d\n",
           WTERMSIG(status));
}

foreground_pid = -1;

printf("Suspension workflow completed.\n");

return 0;
}

```

Compile and run:

```

arikel@DESKTOP-AMUSJME:~/week5prac$ nano task3_sigstsp_suspend.
c
arikel@DESKTOP-AMUSJME:~/week5prac$ gcc task3_sigstsp_suspend.c
-o task3_sigstsp_suspend
arikel@DESKTOP-AMUSJME:~/week5prac$ ./task3_sigstsp_suspend
TASK-3: SIGTSTP Job Suspension and Recovery
Shell PID: 1305
Foreground job PID = 1306
Press Ctrl+Z to suspend the job.
Foreground job started.
Job PID: 1306
Job running... 1/20
Job running... 2/20
Job running... 3/20
Job running... 4/20
Job running... 5/20
^Z
Shell received SIGTSTP.

Job running... 6/20
Shell received SIGTSTP.
Suspending foreground job PID 1306

Job state updated: COMPLETED
Suspension workflow completed.

```

Conclusion: Implemented SIGTSTP handling for suspending a foreground process. The shell captures the suspension request and sends SIGSTOP to the foreground job. `waitpid()` with `WUNTRACED` detects the stopped state and updates the job information. The stopped process is later resumed using `SIGCONT`, after which execution continues until completion.

TASK-4: To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior

Ans:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <termios.h>

int main()
{
    printf("TASK-4: Process Groups and Terminal Control\n");

    pid_t shell_pgid = getpgrp();

    printf("Shell PID   : %d\n", getpid());
    printf("Shell PGID  : %d\n", shell_pgid);

    pid_t child = fork();

    if (child < 0) {
        perror("fork");
        return 1;
    }

    if (child == 0) {

        /*
         * Create a new process group.
         */
        setpgid(0, 0);

        printf("\nChild process created.\n");
        printf("Child PID   : %d\n", getpid());
    }
}
```

^G Help  
^X Exit

^O Write Out  
^R Read File

^F Where Is  
^\_ Replace

^K Cut  
^U Paste

^T Execute  
^J Justify

```

    printf("Child PID   : %d\n", getpid());
    printf("Child PGID  : %d\n", getpgid());

    for (int i = 1; i <= 8; i++) {
        printf("Foreground process group running... %d/8\n",
              i);
        sleep(1);
    }

    printf("Child process completed.\n");
    exit(0);
}

/*
 * Parent assigns child to its own process group.
 */
setpgid(child, child);

printf("\nParent assigned child to process group %d\n",
      child);

sleep(1);

/*
 * Transfer terminal ownership to child group.
 */
printf("Transferring terminal control to child group...\n");

if (tcsetpgrp(STDIN_FILENO, child) == -1) {
    perror("tcsetpgrp");
}
else {
    printf("Terminal control transferred.\n");
}

```

```

    else {
        printf("Terminal control transferred.\n");
    }

    /*
     * Send a signal to the complete process group.
     */
    printf("Process group ID = %d\n", child);
    printf("Process group is now foreground.\n");

    int status;

    waitpid(child, &status, WUNTRACED);

    /*
     * Restore terminal ownership.
     */
    printf("\nRestoring terminal control to shell...\n");

    if (tcsetpgrp(STDIN_FILENO, shell_pgid) == -1) {
        perror("tcsetpgrp");
    }
    else {
        printf("Terminal control restored.\n");
    }

    if (WIFEXITED(status)) {
        printf("Child group state: COMPLETED\n");
    }
    else if (WIFSIGNALED(status)) {
        printf("Child group terminated by signal %d\n",
            WTERMSIG(status));
    }
}

```

```

    else if (WIFSIGNALED(status)) {
        printf("Child group terminated by signal %d\n",
            WTERMSIG(status));
    }
    else if (WIFSTOPPED(status)) {
        printf("Child group state: STOPPED\n");
    }

    printf("\nProcess group and terminal control test completed>
return 0;
}

```

Compile and run:

```
arikel@DESKTOP-AMUSJME:~/week5prac$ nano task4_process_groups.c
arikel@DESKTOP-AMUSJME:~/week5prac$ gcc task4_process_groups.c
-o task4_process_groups
arikel@DESKTOP-AMUSJME:~/week5prac$ ./task4_process_groups
TASK-4: Process Groups and Terminal Control
Shell PID : 1447
Shell PGID : 1447

Parent assigned child to process group 1448

Child process created.
Child PID : 1448
Child PGID : 1448
Foreground process group running... 1/8
Transferring terminal control to child group...
Terminal control transferred.
Process group ID = 1448
Process group is now foreground.
Foreground process group running... 2/8
Foreground process group running... 3/8
Foreground process group running... 4/8
Foreground process group running... 5/8
Foreground process group running... 6/8
Foreground process group running... 7/8
Foreground process group running... 8/8
Child process completed.

Restoring terminal control to shell...

[2]+  Stopped                  ./task4_process_groups
arikel@DESKTOP-AMUSJME:~/week5prac$ |
```